

CO215 Advanced Programming Laboratory

Spring Semester 2026

Assignment	Assignment 7 – Activity Log Analyzer in Python
Roll Number	CSB24073
Name	Gaurav Nath
Session / Year	Spring / 2026

1. Aim

To develop an Activity Log Analyzer in Python that processes a list of activity records — each containing a user identifier, an action (such as an app or website visited), and its duration — using Python built-in containers and functional programming constructs such as `reduce()`, `defaultdict`, `set/list` comprehensions, and `sorted()` with key functions.

2. Problem Description

Given a list of activity log records of the form:

```
{ "user": str, "action": str, "duration": float }
```

The program must implement three core functions:

- `total_time_per_user(logs)` — computes the total screen time for each user using `reduce()`.
- `most_active_users(logs, k)` — returns the top-k users ranked by total time using `sorted()` with a key.
- `unique_actions(logs)` — returns the set of all distinct actions seen across all log records.

3. Requirements

The implementation must satisfy the following constraints:

- Use dict, set, and list as primary data containers.
- Use comprehensions (set comprehension for `unique_actions`).
- Use `sorted()` with `key=lambda` for ranking users.
- Avoid explicit loops where possible — rely on functional constructs.
- Use typing annotations (`List[Dict]`, `Dict[str, float]`, `Set[str]`).
- Use `defaultdict` optionally for accumulation.
- Use `reduce()` from `functools` to compute total activity time.

4. Algorithm

4.1 `total_time_per_user`

- Initialize a `defaultdict(float)` as accumulator.
- Define a reducer function that adds `log['duration']` to `acc[log['user']]`.
- Apply `reduce()` over the `logs` list with the reducer and accumulator.
- Return the result cast to a plain dict.

4.2 most_active_users

- Call `total_time_per_user()` to get per-user totals.
- Sort the items by duration in descending order using `sorted(..., key=lambda x: x[1], reverse=True)`.
- Return the first `k` user names from the sorted result via a list comprehension.

4.3 unique_actions

- Use a set comprehension over the logs list to extract `log['action']` for every record.
- Return the resulting set — duplicates are automatically eliminated.

5. Python Implementation

File: `activityLogAnalyzer.py`

```
from typing import List, Dict, Set
from collections import defaultdict
from functools import reduce

# 1. Total Time Per User
def total_time_per_user(logs: List[Dict]) -> Dict[str, float]:

    user_time = defaultdict(float)
    def reducer(acc, log):
        acc[log["user"]] += log["duration"]
        return acc

    return dict(reduce(reducer, logs, user_time))

# 2. Most Active Users
def most_active_users(logs: List[Dict], k: int) -> List[str]:

    user_time = total_time_per_user(logs)
    sorted_users = sorted(
        user_time.items(),
        key=lambda x: x[1],
        reverse=True
    )

    return [user for user, _ in sorted_users[:k]]

# 3. Unique Actions
def unique_actions(logs: List[Dict]) -> Set[str]:

    return {log["action"] for log in logs}

# Data
logs = [
    {"user": "Ram", "action": "YouTube", "duration": 25.5},
    {"user": "Mohan", "action": "Instagram", "duration": 10.0},
    {"user": "Mohan", "action": "WhatsApp", "duration": 5.0},
    {"user": "Ram", "action": "YouTube", "duration": 40.0},
    {"user": "Akash", "action": "Facebook", "duration": 15.0},
]
```

```
print("Total Time Per User:")
print(total_time_per_user(logs))

print("\nTop 2 Most Active Users:")
print(most_active_users(logs, 2))

print("\nUnique Actions:")
print(unique_actions(logs))
```

6. Input

The input is a hardcoded list of activity log dictionaries passed to each function:

```
logs = [
    {"user": "Ram", "action": "YouTube", "duration": 25.5},
    {"user": "Mohan", "action": "Instagram", "duration": 10.0},
    {"user": "Mohan", "action": "WhatsApp", "duration": 5.0 },
    {"user": "Ram", "action": "YouTube", "duration": 40.0},
    {"user": "Akash", "action": "Facebook", "duration": 15.0},
]
```

7. Output

7.1 Terminal Output (Text)

```
Total Time Per User:
{'Ram': 65.5, 'Mohan': 15.0, 'Akash': 15.0}

Top 2 Most Active Users:
['Ram', 'Mohan']

Unique Actions:
{'YouTube', 'Instagram', 'WhatsApp', 'Facebook'}
```

7.2 Screenshot

```
New Activity LOG > activityLogAnalyzer.py > ...
1  from typing import List, Dict, Set
2  from collections import defaultdict
3  from functools import reduce
4
5
6  # 1. Total Time Per User
7  def total_time_per_user(logs: List[Dict]) -> Dict[str, float]:
8
9      user_time = defaultdict(float)
10     def reducer(acc, log):
11         acc[log["user"]] += log["duration"]
12         return acc
13
14     return dict(reduce(reducer, logs, user_time))
15
16
17 # 2. Most Active Users
18 def most_active_users(logs: List[Dict], k: int) -> List[str]:
19
20     user_time = total_time_per_user(logs)
21     sorted_users = sorted(
22         user_time.items(),
23         key=lambda x: x[1],
24         reverse=True
25     )
26
27     return [user for user, _ in sorted_users[:k]]
28
```

```

18 def most_active_users(logs: List[Dict], k: int) -> List[str]:
23     key=lambda x: x[1],
24     reverse=True
25 )
26
27     return [user for user, _ in sorted_users[:k]]
28
29
30 # 3. Unique Actions
31 def unique_actions(logs: List[Dict]) -> Set[str]:
32
33     return {log["action"] for log in logs}
34
35
36 #Data
37 logs = [
38     {"user": "Ram", "action": "YouTube", "duration": 25.5},
39     {"user": "Mohan", "action": "Instagram", "duration": 10.0},
40     {"user": "Mohan", "action": "WhatsApp", "duration": 5.0},
41     {"user": "Ram", "action": "YouTube", "duration": 40.0},
42     {"user": "Akash", "action": "Facebook", "duration": 15.0},
43 ]
44
45
46 print("Total Time Per User:")
47 print(total_time_per_user(logs))
48
49 print("\nTop 2 Most Active Users:")
50 print(most_active_users(logs, 2))
51
52 print("\nUnique Actions:")
53 print(unique_actions(logs))

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Code + - [] []

```

PS D:\Advanced Programming> python -u "d:\Advanced Programming\New Activity LOG\activityLogAnalyzer.py"
Total Time Per User:
{'Ram': 65.5, 'Mohan': 15.0, 'Akash': 15.0}

Top 2 Most Active Users:
['Ram', 'Mohan']

Unique Actions:
{'Facebook', 'WhatsApp', 'Instagram', 'YouTube'}
PS D:\Advanced Programming>

```

8. Learning Outcomes

- Understood the use of `reduce()` from `functools` to accumulate values across a list without explicit loops, applying it to compute total duration per user.
- Practiced the use of `defaultdict` to simplify accumulator initialization, avoiding `KeyError` and making the reducer function cleaner.
- Applied `sorted()` with a `lambda` key and `reverse=True` to rank users by total activity time, reinforcing the use of higher-order functions in Python.
- Used set comprehensions to extract unique actions from logs in a single expressive line, highlighting the power of Python comprehensions.
- Reinforced typing annotations (`List[Dict]`, `Dict[str, float]`, `Set[str]`) for writing readable, maintainable, and type-safe Python code.